Decentralized Multi-Robot Connectivity Control: How It Actually Works

A Deep Dive into Lambda-2 Estimation and Control Barrier Functions

Executive Summary

This document explains how a team of robots maintains network connectivity while navigating to their goals. Each robot independently estimates how well-connected the network is (using a metric called lambda-2) and adjusts its behavior to prevent the team from splitting apart. The system uses multi-hop communication, position prediction, and consensus mechanisms to work in a fully decentralized manner.

1. The Problem We're Solving

Imagine a team of robots exploring an environment. Each robot has:

- A goal location it wants to reach
- Limited communication range (can only talk to nearby robots)
- Obstacles to avoid
- Need to avoid collisions with teammates

The Challenge: If robots just go straight to their goals, the team might split into disconnected groups. Once disconnected, they can't coordinate or share information. We need the robots to balance two competing goals:

1. Reach their individual destinations efficiently
2. Stay connected as a team

Key Constraint: There's no central controller. Each robot must make its own decisions based only on what it can sense and what it hears from neighbors.

2. Understanding Network Connectivity: Lambda-2

2.1 The Graph Representation

We model the robot network as a graph:

- Each robot is a **node**
- Two robots have an **edge** between them if they're within communication range
- Edge **weights** get stronger as robots get closer

2.2 The Laplacian Matrix

From this graph, we build the Laplacian matrix L:

$$L = D - A$$

Where:

- A = Adjacency matrix (edge weights between robots)
- D = Degree matrix (sum of each robot's connections)

2.3 What Lambda-2 Tells Us

When we compute the eigenvalues of L and sort them:

$$0 = \lambda_1 < \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n$$

Lambda-2 (the second smallest eigenvalue) is called 'algebraic connectivity':

- **Lambda-2 = 0:** Network is disconnected (team is split)
- **Lambda-2 > 0:** Network is connected
- **Larger lambda-2:** Stronger connectivity, more resilient to link failures
- **Smaller lambda-2:** Weak connectivity, close to breaking apart

Why This Matters: By monitoring lambda-2, each robot can tell if the network is about to fragment. If lambda-2 drops below a threshold (lambda2_warn), robots know they need to prioritize connectivity over reaching their goals.

3. The Estimation Challenge

3.1 What Each Robot Knows

Here's the key problem: To compute lambda-2, a robot needs to know where ALL robots are. But each robot only has direct knowledge of:

- **Itself:** Perfect knowledge of its own position and velocity
- **Direct neighbors:** Robots within communication range R_glob

For robots beyond communication range, the robot must ESTIMATE their positions. This is where the complexity begins.

3.2 Three Types of Knowledge

Type 1: Perfect Knowledge (100% confidence)

- Self and direct neighbors
- Measured directly with sensors
- No uncertainty

Type 2: Communicated Knowledge (High confidence, decays with time/hops)

- Information received through multi-hop communication
- Includes position, velocity, and timestamp
- Quality degrades with age and number of hops

Type 3: Estimated Knowledge (Low confidence)

- For robots with no recent communication
- Blends starting position with goal position
- Assumes robots are progressing toward their goals

4. Multi-Hop Communication: How Information Spreads

4.1 The Basic Concept

Multi-hop communication allows information to travel beyond direct communication range. Think of it like a relay race: Robot A tells Robot B, who tells Robot C, and so on.

4.2 The Message Structure

Each message contains:

- **robot_id:** Which robot this information is about
- **position:** [x, y] coordinates
- **velocity:** [vx, vy] velocity vector
- **timestamp:** When this information was created
- **hop_count:** How many times this message has been forwarded

4.3 How Multi-Hop Works Each Timestep

Step 1: Self-Update

Each robot updates its own entry in its knowledge base with current position, velocity, and timestamp. This ensures the robot always has fresh information about itself.

Step 2: Message Exchange

Each robot sends its ENTIRE knowledge base to all direct neighbors. This means:

- Robot A knows about robots 1, 3, 5
- Robot A broadcasts all this info to its neighbors
- Robot B receives this and learns about robots 1, 3, 5 (even if they're far away)

Step 3: Message Processing and Filtering

When a robot receives messages from neighbors, it decides whether to accept each piece of information:

Acceptance Criteria:

A robot accepts a message if:

1. It's NEW information (robot didn't know about this robot before), AND
2. hop_count <= max_hops (default: N, the number of robots)

OR

1. It's NEWER information (timestamp > existing timestamp), AND
2. hop_count <= max_hops

When accepting a forwarded message, the robot increments the hop_count by 1.

Step 4: Stale Data Cleanup

After processing messages, each robot removes information older than position_timeout (1.0 second). This prevents using outdated position estimates.

4.4 Example: Information Propagating Through Network

Scenario: 4 robots in a line, each within range of its immediate neighbors:

R1 ---- R2 ---- R3 ---- R4

Timestep 0:

- R1 knows: {R1, R2}
- R2 knows: {R1, R2, R3}
- R3 knows: {R2, R3, R4}
- R4 knows: {R3, R4}

Timestep 1 (after one round of communication):

- R1 receives info about R3 from R2 (hop_count=2)
- R1 now knows: {R1, R2, R3}
- R4 receives info about R2 from R3 (hop_count=2)
- R4 now knows: {R2, R3, R4}

Timestep 2 (after two rounds):

- R1 receives info about R4 from R2 (hop_count=3)
- R1 now knows: {R1, R2, R3, R4} - complete knowledge!
- Similarly, R4 learns about R1

Key Insight: In a connected network, information propagates outward like ripples. After enough timesteps, every robot eventually learns about every other robot, though with varying confidence based on hop count and age.

5. Velocity Extrapolation: Predicting Robot Positions

5.1 The Problem with Delays

When Robot A receives information about Robot B through multi-hop communication, that information might be old (e.g., 0.3 seconds). If we use the old position directly, our estimate will be inaccurate.

5.2 The Solution: Velocity-Based Prediction

If the message includes velocity, we can predict where the robot is NOW:

$$\mathbf{x}_{\text{current}} = \mathbf{x}_{\text{received}} + \mathbf{v} \cdot \Delta t$$

Where delta-t is the time since the message was created (current_time - timestamp).

5.3 Confidence Decay in Extrapolation

Extrapolation becomes less reliable as time passes because:

1. Robots might change velocity (turn, accelerate, stop)
2. Faster robots accumulate more positional error

Normalized Age Calculation:

To make confidence decay dimensionless and transferable across scenarios:

$$\text{normalized\_age} = \frac{v_{\text{char}} \cdot \Delta t}{R_{\text{glob}}}$$

This captures the idea: 'How far could the robot have moved relative to the communication range?'

Confidence Formula:

$$\text{conf} = 90 \cdot e^{-2 \cdot \text{normalized\_age}} \cdot e^{-0.1 \cdot \text{hop\_count}}$$

Confidence starts at 90% (not 100% because it's not direct observation) and decays exponentially with both age and hop count.

5.4 Extrapolation Limits

extrapolation_max_age = 0.5 seconds

If information is older than this, extrapolation is disabled. The system falls back to using the stale position directly (with reduced confidence) because extrapolation becomes too unreliable.

6. Blended Estimation: When Communication Fails

6.1 The Fallback Strategy

When a robot has NO recent communication about another robot (older than 1 second or never heard), it must make an educated guess. The system blends two pieces of information:

1. **Starting Position (SP):** Where the robot was at the beginning
2. **Goal Position (GP):** Where the robot is trying to go

6.2 Time-Based Confidence Shift

As time passes, the robot is more likely to have moved toward its goal:

Starting Position Confidence (decays over time):

$$\text{SP\_conf} = 100 \cdot e^{-t/3.0} + 1$$

At t=0: SP confidence = 100% (robot is definitely at start)

At t=3: SP confidence ≈ 37% (might have moved)

At t=9: SP confidence ≈ 6% (probably far from start)

Goal Position Confidence (grows over time):

$$\text{GP\_conf} = 101 - \text{SP\_conf}$$

At t=0: GP confidence ≈ 1% (robot just started)

At t=3: GP confidence ≈ 64% (making progress)

At t=9: GP confidence ≈ 95% (probably near goal)

6.3 Weighted Blend

The estimated position is a confidence-weighted average:

$$\mathbf{x}_{\text{est}} = \frac{\text{SP\_conf} \cdot \mathbf{x}_{\text{start}} + \text{GP\_conf} \cdot \mathbf{x}_{\text{goal}}}{\text{SP\_conf} + \text{GP\_conf}}$$

Early in the mission: estimate is close to starting position
Late in the mission: estimate is close to goal position
Middle of mission: estimate is somewhere in between

6.4 Why This Works

This is a model-based estimate that assumes robots behave rationally (move toward goals). While not perfect, it's better than having no estimate at all. The low confidence (typically 20-40%) reflects this uncertainty.

7. Building the Connectivity Graph

7.1 From Positions to Adjacency Matrix

Once each robot has estimated positions for all others, it builds a connectivity graph:

Step 1: Calculate Distances

For every pair of robots (i, j), compute:

$$d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|$$

Step 2: Apply Communication Range Threshold

If d_ij > R_glob: no edge (robots can't communicate)
If d_ij <= R_glob: create weighted edge

Step 3: Calculate Edge Weight (Smooth Connectivity Function)

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{R_{glob}^2 - d_{ij}^2 + \varepsilon}\right)$$

This function has nice properties:

- w $\to$ 1 as d $\to$ 0 (robots very close = strong connection)
- w $\to$ 0 as d $\to$ R_glob (robots at edge of range = weak connection)
- Smooth gradient (differentiable for control)

7.2 Phantom Edge Filtering

The Problem: Low-confidence position estimates can create 'phantom edges' - connections that don't actually exist but appear in the graph due to poor estimates.

The Solution: Before adding an edge between robots i and j, verify:

1. Both robots have confidence >= edge_conf_threshold (60%)
2. At least one of the robots is 'verified' (self, direct neighbor, or known through communication)

This prevents the graph from including edges between two poorly-estimated robots, which could artificially inflate lambda-2.

7.3 Finding Connected Components

After building the adjacency matrix, use Breadth-First Search (BFS) to find which robots are in the same connected component as the ego robot:

BFS Algorithm:

1. Start with ego robot, mark as visited
2. Add ego robot to queue
3. While queue not empty:

- Remove robot from queue

- For each neighbor with edge weight > 0:

- If not visited, mark visited and add to queue

4. All visited robots form the connected component

Why we need this: Lambda-2 is only meaningful for the component containing the ego robot. Including disconnected robots would give incorrect results.

8. Computing Lambda-2

8.1 The Laplacian Matrix

For the connected component, construct the Laplacian:

Step 1: Extract Subgraph

If component has robots {2, 5, 7, 9}, extract the 4×4 submatrix from the full adjacency matrix.

Step 2: Calculate Degree Matrix

D is diagonal, where D_ii = sum of all edge weights for robot i:

$$D_{ii} = \sum_{j=1}^{n} A_{ij}$$

Step 3: Form Laplacian

$$L = D - A$$

8.2 Eigenvalue Computation

Use MATLAB's eig() function to find all eigenvalues of L. Sort them in ascending order:

$$\lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_n$$

Extract lambda_2 (the second value in the sorted list).

8.3 Special Cases

Component size = 1 (robot is isolated): lambda-2 = 0

The robot has no connections. Network is disconnected.

Component size >= 2: Compute normally

Even with just 2 robots, we can compute lambda-2. It represents the strength of that single connection.

8.4 Confidence in Lambda-2 Estimate

The confidence in the lambda-2 estimate depends on:

1. **Position confidence:** Average confidence of all robots in component
2. **Component size:** Larger component = more complete view = higher confidence

Formula:

$$\text{lambda2\_conf} = \text{avg\_pos\_conf} \cdot \frac{n_{comp}}{N}$$

If the robot can see 8 out of 10 robots with average 85% position confidence:

lambda2_conf = 0.85 × (8/10) = 68%

9. Validation: Catching Estimation Errors

9.1 Why Validation is Needed

Position estimation errors can lead to incorrect lambda-2 values. Three heuristic checks catch common problems:

9.2 Check 1: Eigenvalue Bounds Violation

Mathematical Property: For an n-node graph, lambda-2 <= n-1

Test: If lambda-2 > n-1 + small_tolerance (0.01)

Diagnosis: Numerical error or severe position estimation problem

Action: Flag this estimate as invalid

9.3 Check 2: Suspicious Disconnection

Situation: lambda-2 ≈ 0 (disconnected) but robot has neighbors

Test: If lambda-2 < 0.001 AND local_degree > 0

Diagnosis: Something is wrong. If the robot has neighbors, lambda-2 should be > 0.

Likely Cause: Phantom edge filtering removed all remote connections, or BFS failed

9.4 Check 3: Weak Spectral Gap

Situation: lambda-2 and lambda-3 are very close

Test: If lambda-2 > 0.8 × lambda-3

Diagnosis: Weak spectral gap can indicate numerical instability or nearly-disconnected graph

Interpretation: Not necessarily wrong, but worth monitoring

9.5 Validation Frequency

Checks run every **validation_frequency** timesteps (default: 5) to avoid computational overhead. When a check fails, a flag is set and a warning symbol appears in visualization.

10. Consensus: Combining Multiple Estimates

10.1 The Motivation

Each robot computes its own lambda-2 estimate based on its local view. Different robots will get slightly different values because they have different information. Consensus allows robots to refine their estimates by consulting neighbors.

10.2 The Consensus Algorithm

Step 1: Collect Neighbor Estimates

Each robot gathers lambda-2 estimates from all direct neighbors (within R_glob).

Step 2: Calculate Weights

Each neighbor's estimate is weighted by:

$$w_j = \text{confidence}_j \cdot \text{degree}_j$$

Why degree matters: A robot with more connections likely has a more complete view of the network.

Step 3: Outlier Rejection

Calculate weighted mean and standard deviation of neighbor estimates:

$$\mu = \sum_j w_j \lambda_{2,j}, \quad \sigma = \sqrt{\sum_j w_j (\lambda_{2,j} - \mu)^2}$$

Reject estimates where |lambda_2 - mu| > 2×sigma (outlier_threshold = 2.0)

Step 4: Blend Own Estimate with Consensus

After removing outliers, compute consensus estimate and blend:

$$\lambda_{2,\text{refined}} = 0.7 \cdot \lambda_{2,\text{own}} + 0.3 \cdot \lambda_{2,\text{consensus}}$$

The 70/30 split means: 'Trust your own estimate more, but incorporate neighbor information.'

10.3 Why Consensus Helps

- **Reduces impact of individual errors:** If one robot has bad estimates, consensus dampens it

- **Incorporates different viewpoints:** Different robots see different parts of the network

- **Maintains decentralization:** No global coordinator needed

10.4 Minimum Neighbors Requirement

Consensus only runs if robot has >= **min_consensus_neighbors** (default: 2). With fewer neighbors, consensus is unreliable and own estimate is used directly.

11. Control Barrier Functions: Safe Control Synthesis

11.1 The Control Problem

Each robot wants to reach its goal but must satisfy safety constraints:

1. Don't collide with teammates
2. Don't collide with obstacles
3. Maintain connectivity with neighbors (if lambda-2 is low)

11.2 Nominal Control

Without constraints, each robot would use a simple PD controller to reach its goal:

$$\mathbf{u}_{\text{nom}} = -k_p(\mathbf{x} - \mathbf{x}_{\text{goal}}) - k_d\mathbf{v}$$

This is a spring-damper system: position error creates force toward goal, velocity damping prevents overshoot.

11.3 Control Barrier Functions (CBFs)

A CBF is a function h(x) that ensures safety. The key property:

$$\frac{dh}{dt} + \alpha(h) \geq 0$$

If this inequality is satisfied, the safety set {x : h(x) >= 0} remains forward invariant (system stays safe).

11.4 Collision Avoidance CBF

For robots i and j, define safety function:

$$h_{\text{col}} = \|\mathbf{x}_i(t+T) - \mathbf{x}_j(t+T)\|^2 - d_{\text{min}}^2$$

Where x(t+T) is predicted position T_pred seconds ahead:

$$\mathbf{x}(t+T) = \mathbf{x}(t) + T \cdot \mathbf{v}(t)$$

Taking time derivative and applying CBF condition gives linear constraint on u_i:

$$-2T\mathbf{x}_{ij}^T\mathbf{u}_i \leq b_{\text{col}}$$

This constraint prevents robot i from choosing a control that would lead to collision with j.

11.5 Connectivity Maintenance CBF

To maintain links with nearby neighbors, define:

$$h_{\text{conn}} = R_{\text{loc}}^2 - \|\mathbf{x}_i(t+T) - \mathbf{x}_j(t+T)\|^2$$

This is the 'opposite' of collision avoidance: we want to stay CLOSE to neighbors. Applied when distance < R_loc + connectivity_margin.

11.6 Adaptive CBF Gain Based on Lambda-2

Key Insight: When lambda-2 is low (network weakly connected), we need STRONGER connectivity enforcement.

Conservative Lambda-2:

$$\lambda_{2,\text{cons}} = \max(0, \lambda_{2,\text{est}} - 2\sigma)$$

Use a conservative estimate (subtract 2× standard deviation) to account for uncertainty.

Gain Adaptation:

If lambda2_cons > lambda2_eps (small threshold, e.g., 0.01):

- If lambda2_cons < lambda2_warn: Increase gain

$$\text{gain}_{\text{eff}} = \text{gain}_{\text{base}} \cdot [1 + k_\lambda \cdot (\lambda_{2,\text{warn}} - \lambda_{2,\text{cons}})]$$

- If lambda2_cons >= lambda2_warn: Use base gain (network is healthy)
- Additionally increase gain if uncertainty is high (std > 0.1)

Result: When network is weak or uncertain, robots prioritize maintaining connectivity over reaching goals.

11.7 Quadratic Program (QP) Formulation

Combine all constraints into a QP:

Objective: Stay close to nominal control

$$\min_{\mathbf{u}} \|\mathbf{u} - \mathbf{u}_{nom}\|^2$$

Subject to:

- Collision avoidance constraints (one per nearby robot)
- Connectivity maintenance constraints (one per close neighbor)
- Obstacle avoidance constraints (one per obstacle)

MATLAB's quadprog solves this QP to find the control u that satisfies all safety constraints while staying as close as possible to the desired nominal control.

11.8 Infeasibility Handling

If the QP has no solution (constraints are contradictory), fall back to nominal control. This is rare but can happen in extreme situations (e.g., robot trapped between obstacle and other robots).

12. Putting It All Together: The Complete Loop

12.1 Simulation Timestep Flow

Phase	What Happens	Key Outputs
1. Communication	Robots share knowledge bases, messages propagate (multi-hop)	Updated knowledge{i} for each robot
2. Position Estimation	Each robot estimates positions of all others using observations, communication, or dead reckoning	positions_estimated, velocities_estimated, confidences, uncertainties
3. Graph Construction	Build adjacency matrix with phantom edge filtering, force connected components	A_fiedler, connected_components, n_edges
4. Lambda-2 Computation	Compute Laplacian eigenvalues for connected component	lambda2_estimates, lambda2_confidence, lambda2_std
5. Validation	Apply heuristic checks every N steps	validation_flags (0=OK, 1-3=issues)
6. Consensus	Refine estimates using weighted neighbor consensus (if validation rejects)	lambda2_consensus (refined values)
7. Control Synthesis	Compute conservative lambda-2, adapt CBF gains, solve QP for safe controls	U (control inputs for all robots)
8. Dynamics	Integrate dynamics: v += dt×U, x += dt×v, apply velocity saturation	Updated x, v (new positions, velocities)
9. Logging	Record lambda-2, confidence, validation flags, trajectories	lambda2_log, lambda2_conf_log, xlog
10. Visualization	Render network graph, robots, obstacles, warnings	Updated figure (every 5 steps)

12.2 Information Flow Diagram

Robot i's decision-making process:

Inputs:

- Own state: x_i, v_i (perfect)
- Direct neighbors: x_j, v_j for j in neighbors (perfect)
- Communicated data: messages from multi-hop (timestamped, hop-counted)
- Historical data: x_start, x_goal, t_start

Processing:

1. Fuse information → position estimates + confidences
2. Build graph → adjacency matrix
3. Compute connectivity → lambda-2 ± uncertainty
4. Validate + consensus → refined lambda-2
5. Adapt control parameters → CBF gains
6. Solve QP → safe control u_i

Outputs:

- Control input: u_i (2D vector)
- State broadcast: (x_i, v_i, t, hop=0) sent to neighbors

13. Critical Thresholds and Their Roles

13.1 Communication Thresholds

Parameter	Default	Purpose
R_glob	varies	Communication range - robots within this distance can exchange message
max_hops	N	Maximum times a message can be forwarded - prevents infinite loops
position_timeout	1.0 sec	Information older than this is discarded as stale
extrapolation_max_age	0.5 sec	Beyond this, velocity extrapolation disabled (too uncertain)

13.2 Confidence Thresholds

Parameter	Default	Purpose
edge_conf_threshold	60%	Minimum confidence to include edge in graph - prevents phantom edges
sp_conf_max	100	Maximum starting position confidence (at t=0)
sp_conf_min	1	Minimum starting position confidence (as $t \to \infty$)
outlier_threshold	2.0 σ	Reject consensus estimates beyond this many std deviations

13.3 Lambda-2 Control Thresholds

Parameter	Default	Purpose
lambda2_eps	0.01	Below this, network considered disconnected (trigger strong action)
lambda2_warn	varies	Warning threshold - below this, increase CBF connectivity gains
k_lambda_glob	varies	Gain scaling factor - how aggressively to respond to low lambda-2

13.4 Safety Thresholds

Parameter	Default	Purpose
d_min	varies	Minimum allowed distance between robots (collision threshold)
R_loc	varies	Local connectivity radius - maintain links within this distance
conn_margin	varies	Buffer: start enforcing connectivity before reaching R_loc
rsafe	varies	Safety margin around obstacles
v_max	2.0 m/s	Maximum velocity - enforced via saturation

14. Optional: Monte Carlo Uncertainty Quantification

14.1 The Concept

When enabled (CONFIG.mc_enabled = true), the system uses Monte Carlo sampling to rigorously quantify uncertainty in lambda-2 estimates.

14.2 The Algorithm

For each robot, repeat mc_samples times (default: 50):

Step 1: Sample Positions

For each robot j, sample from Gaussian distribution:

$$\mathbf{x}_j^{\text{sample}} \sim \mathcal{N}(\mathbf{x}_j^{\text{est}}, \sigma_j^2)$$

Step 2: Build Adjacency Matrix

Using sampled positions, construct adjacency matrix as normal.

Step 3: Compute Lambda-2

Find connected component and compute lambda-2 for this sample.

Step 4: Store Result

Add lambda-2 value to samples array.

14.3 Statistical Analysis

After all samples collected:

$$\lambda_{2,\text{mean}} = \frac{1}{M}\sum_{i=1}^{M}\lambda_{2,i}$$

$$\lambda_{2,\text{std}} = \sqrt{\frac{1}{M-1}\sum_{i=1}^{M}(\lambda_{2,i} - \lambda_{2,\text{mean}})^2}$$

14.4 Enhanced Confidence Metric

With Monte Carlo, confidence incorporates estimation consistency:

$$\text{CV} = \frac{\lambda_{2,\text{std}}}{\lambda_{2,\text{mean}}}$$

$$\text{mc\_conf} = e^{-2 \cdot \text{CV}}$$

High CV (high variability) → low confidence
Low CV (consistent estimates) → high confidence

14.5 Computational Cost

Monte Carlo multiplies computation by factor of mc_samples:

- Without MC: $O(N^2)$ adjacency + $O(n^3)$ eigenvalues
- With MC: $50 \times [O(N^2) + O(n^3)]$

Recommendation: Enable only when rigorous uncertainty quantification is essential and computational resources permit. For real-time control, the deterministic method (disabled MC) is usually sufficient.

15. Key Design Principles

15.1 Decentralization

Every robot makes independent decisions based only on local information. No central coordinator means:

- System scales to large teams
- Robust to single-point failures
- Works with limited communication

15.2 Conservative Estimation

When uncertain, the system errs on the side of caution:

- Use conservative lambda-2 (mean - 2σ) for control
- Increase CBF gains when uncertainty is high
- Filter out low-confidence edges (phantom edge removal)

15.3 Multi-Source Information Fusion

The system intelligently combines:

1. Direct sensing (highest trust)
2. Multi-hop communication (medium trust, decays with age/hops)
3. Model-based prediction (lowest trust, blends start/goal)

15.4 Dimensionless Parameters

All decay rates are normalized:

- Time normalized by characteristic timescale
- Distance normalized by R_glob
- Velocity×time normalized by R_glob

Benefit: Parameters transfer across different scenarios without retuning.

15.5 Graceful Degradation

System remains functional even when components fail:

- Communication fails → Use blended estimates
- QP infeasible → Fall back to nominal control
- Validation fails → Flag but continue operating
- Few consensus neighbors → Use own estimate

16. Worked Example Scenario

16.1 Setup

5 robots, R_glob = 3.0 m, starting in a line, goals form a different configuration.

16.2 t = 0.0 seconds (Initialization)

Robot 3's view:

- Self: x_3, v_3 (100% confidence)
- Neighbors: R2 and R4 within range (100% confidence)
- Remote: R1 and R5 outside range

→ R1 estimated at start position (conf=100%)

→ R5 estimated at start position (conf=100%)

- Lambda-2 computation: All 5 in component, lambda-2 = 1.2
- Control: lambda-2 healthy, use normal CBF gains

16.3 t = 2.0 seconds (Mission progressing)

Robot 3's view:

- Self: x_3, v_3 (100%)
- Neighbors: R2, R4 (100%)
- R1: Received via R2, hop_count=1, age=0.1s

→ Extrapolate: x_1 + v_1×0.1s (conf=82%)

- R5: No communication yet

→ Blend: 67% start + 33% goal (conf=35%)

- Lambda-2: Component of 4 robots (R5 excluded due to low confidence)

→ lambda-2 = 0.8 ± 0.1

- Validation: Pass all checks
- Consensus: 2 neighbors have lambda-2 ≈ 0.75, 0.85

→ Refined: 0.7×0.8 + 0.3×0.80 = 0.80

- Control: lambda-2 < lambda2_warn, increase CBF gain by 15%

16.4 t = 5.0 seconds (Critical moment)

Robot 3's view:

- R1: Far away, last comm 2s ago → stale, removed
- R2: Still neighbor (100%)
- R4: Now at edge of range (85%)

- R5: Blended estimate 30% start + 70% goal (conf=28%)
- Lambda-2: Component of 3 robots {R2, R3, R4}

→ lambda-2 = 0.3 ± 0.15

→ Conservative: 0.3 - 2×0.15 = 0.0 (very weak!)

- Control: STRONG connectivity enforcement

→ CBF gain increased 200%

→ Robot slows progress toward goal to maintain links

16.5 t = 8.0 seconds (Recovery)

Robots have adjusted positions to strengthen network:

- All robots now within multi-hop range
- Lambda-2 recovers to 1.1
- CBF gains return to normal
- Robots resume efficient progress to goals

17. Summary and Key Takeaways

17.1 What We Learned

Multi-hop communication allows information to propagate beyond direct sensing range. Messages include hop counts and timestamps to manage quality. Stale information is removed.

Position estimation fuses perfect knowledge (self, neighbors), communicated knowledge (extrapolated with velocity), and model-based estimates (blended start/goal positions).

Confidence tracking uses exponential decay with normalized parameters. Low-confidence edges are filtered to prevent phantom connections.

Lambda-2 estimation requires building adjacency matrix, finding connected components, and computing eigenvalues. Uncertainty is quantified via confidence metrics (or Monte Carlo sampling).

Validation catches common estimation errors using mathematical bounds, consistency checks, and spectral analysis.

Consensus improves estimates by weighted averaging of neighbor opinions with outlier rejection. 70/30 blend maintains trust in own estimate while incorporating neighbor information.

Adaptive control modulates CBF gains based on conservative lambda-2. When connectivity is weak or uncertain, robots prioritize maintaining links over reaching goals.

17.2 Why This Approach Works

The system balances multiple competing objectives:

- Task completion (reaching goals) vs safety (staying connected)
- Optimism (use available information) vs caution (account for uncertainty)
- Decentralization (independent decisions) vs coordination (consensus)

By explicitly modeling uncertainty and adapting control parameters, the system remains robust to estimation errors while achieving efficient coordination.

17.3 Tuning Guidance

For more conservative (safer) behavior:

- Increase edge_conf_threshold (require higher confidence)
- Increase outlier_threshold (allow more variation)
- Increase CBF gains
- Use larger safety margins (conn_margin, rsafe)

For faster convergence:

- Decrease decay rates (maintain confidence longer)
- Increase consensus weight beyond 0.3
- Enable Monte Carlo for better uncertainty estimates

For scalability to larger teams:

- Limit max_hops to prevent message explosion
- Increase validation_frequency to reduce computation
- Disable Monte Carlo (use deterministic estimation)

This document explains the complete operation of a decentralized multi-robot control system. Each robot independently estimates network connectivity, shares information through multi-hop communication, validates and refines estimates through consensus, and adapts its control strategy to maintain both safety and connectivity. The system is designed to be robust, scalable, and effective in real-world conditions with limited communication and uncertain information.